

3. STM32 Development

3. STM32 Development

3.1 Getting Started

3.1.1 Wiring Instruction

3.1.2 Environment Configuration

3.2 Development Example

3.2.1 Reading the Servo Status

3.2.2 Reading the Servo Status

3.2.3 Servo Movement Control in Write Mode

3.2.4 Reading the Servo Status

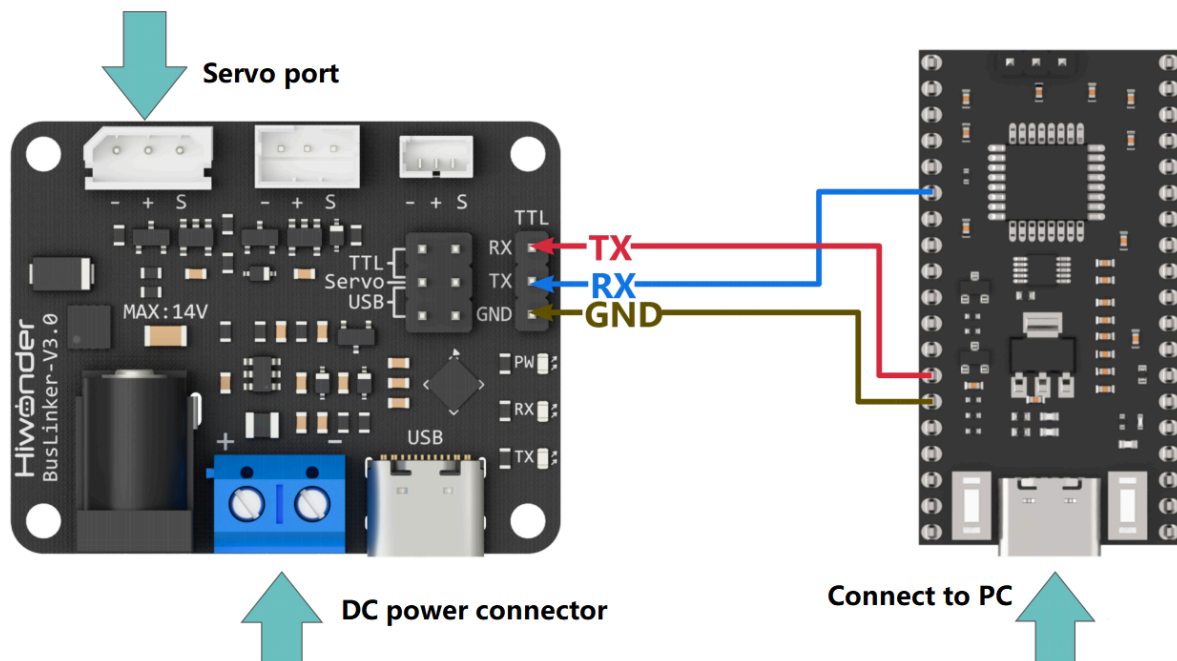
3.2.5 Servo Movement Control in SyncWrite Mode

3.2.6 Reading the Servo Status

3.1 Getting Started

3.1.1 Wiring Instruction

This example uses the STM32 controller for development, with power, program download, and debug data handled via the data cable. The bus servo is connected to the debugging board, which is powered by an 11.1 V lithium battery. The STM32F103RBT6 microcontroller and the debugging board are then connected through the serial pins using Dupont wires.



Note

- When using Hiwonder's lithium battery, connect the battery cable with the red wire to the positive (+) terminal and the black wire to the negative (-) terminal of the DC port.

- Before connecting the battery cables, make sure they are not already attached to the lithium battery. This prevents the risk of a short circuit caused by accidental contact between the positive and negative wires.

3.1.2 Environment Configuration

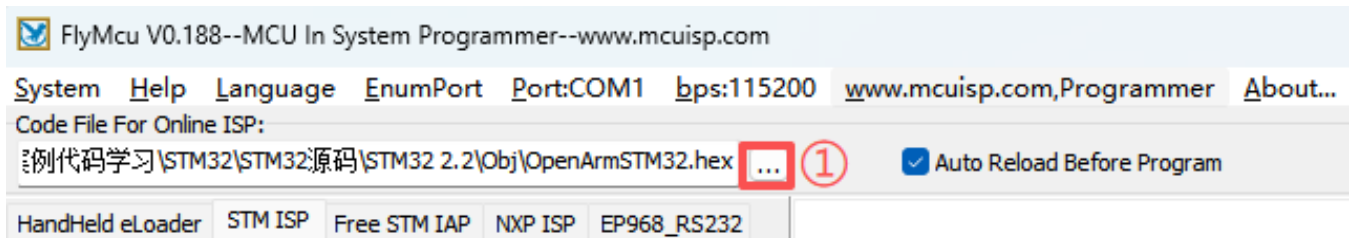
Install Keil MDK IDE software on PC. The software package is stored in [2. Softwares \ STM32 Programming Software](#). For the detailed operations of Keil MDK IDE, please refer to the relevant tutorials.

3.1.2.1 Program Upload

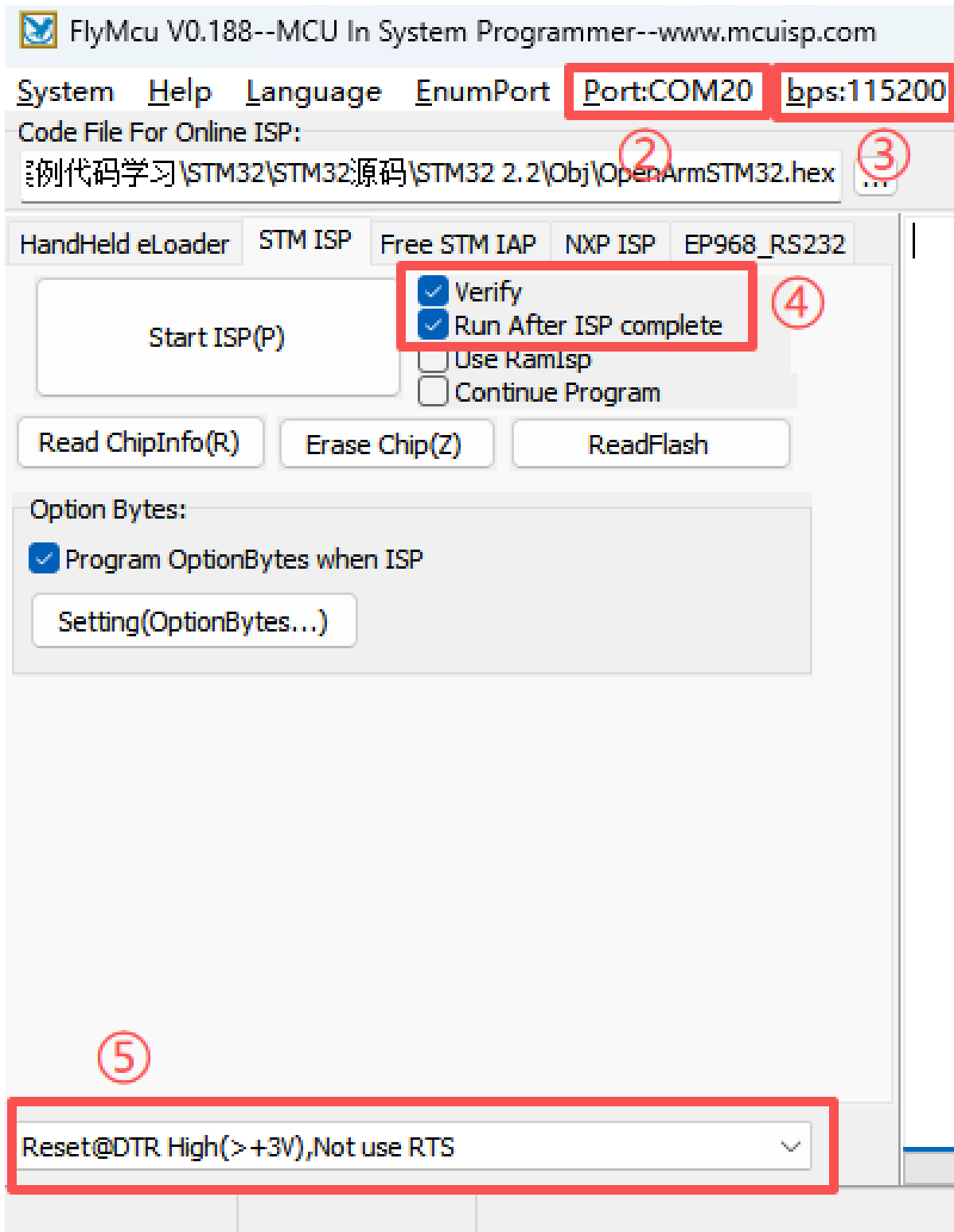
1. Connect the controller to the computer using a USB cable.
2. Double-click to open the STM32 serial programming software FlyMcu.



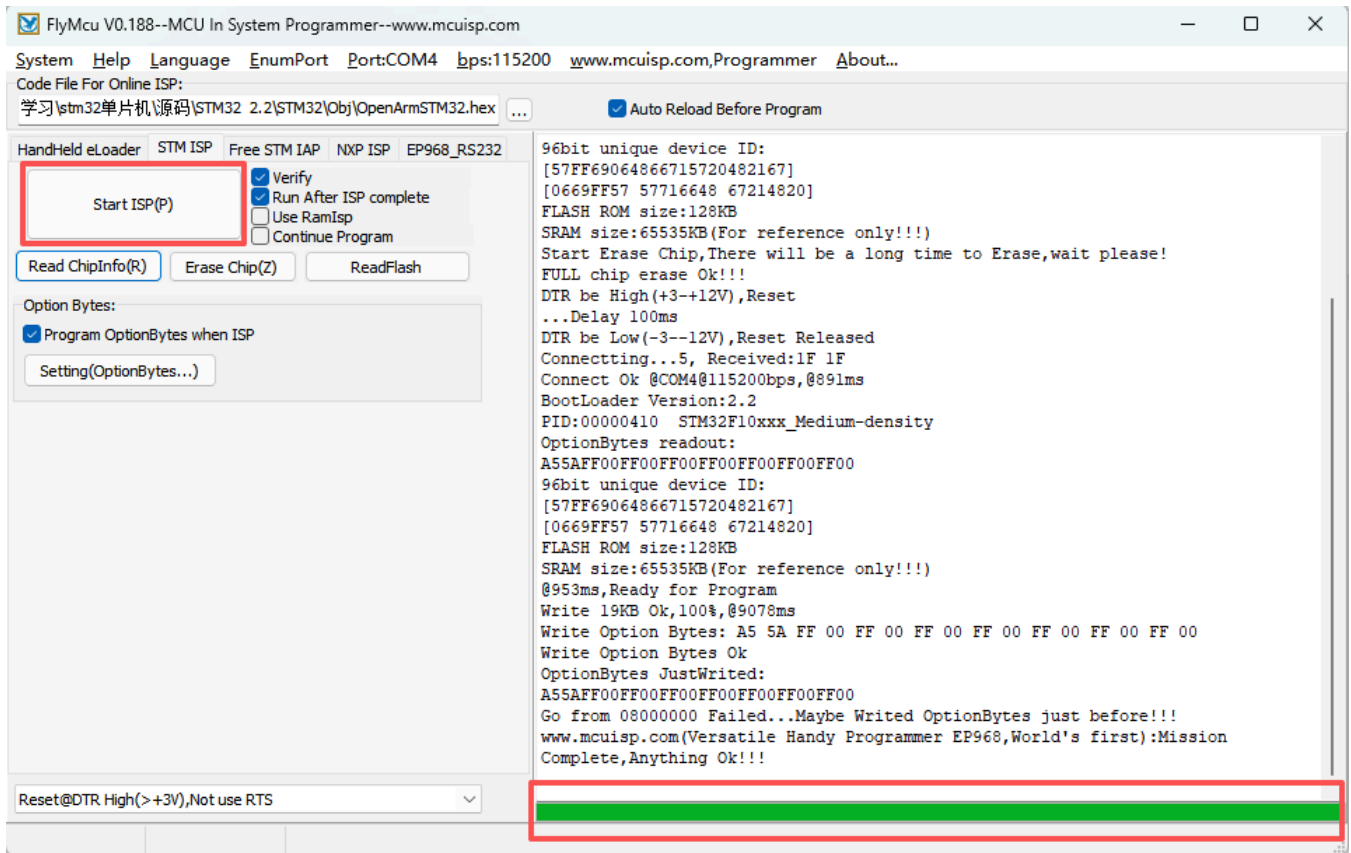
3. Click the ... icon and select the HEX file to be programmed from the path: **Serial_Servo\MDK-ARM\Serial_Servo**.



4. At position ② in the figure, select the serial port corresponding to the controller. At ③, select a baud rate of 115200. Configure ④ and ⑤ according to the options shown in the figure.



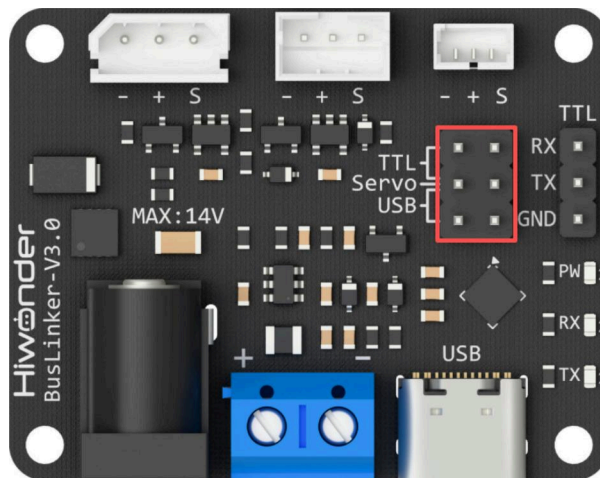
5. Click **Start ISP(P)**, then press the **RST** button on the controller. The program will automatically begin downloading. Once the following content appears in the log output on the right, the program download will be complete.



3.2 Development Example

Note

Before running this example, ensure that the jumpers on the debugging board are installed on the Servo and TTL pins. Otherwise, communication will not function properly.



3.2.1 Reading the Servo Status

This example displays the bus servo status information through the serial debug assistant window.

3.2.1.1 Run Program

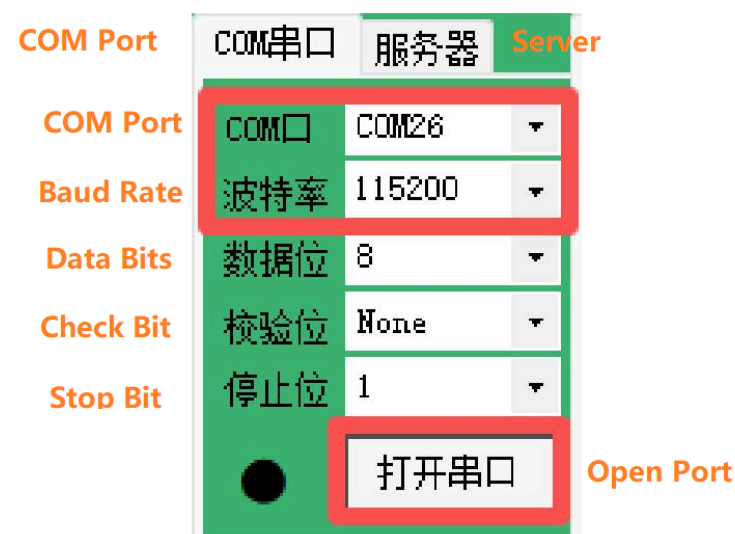
Double-click the **Serial_Servo.uvprojx** file located at the [STM32 Development \ Program \ Serial Servo \ MDK-ARM](#) folder. Change the value of the **exp** macro definition in the **main.c** file to **1**, then compile the project.

```
/* Private define -----*/  
/* USER CODE BEGIN PD */  
#define exp 1
```

Connect the STM32F103RBT6 microcontroller to the computer and upload the compiled HEX file to the microcontroller.

3.2.1.2 Program Outcome

After running the program, open the serial debug assistant, select the corresponding COM port, set the baud rate to 115200, and open the port. The debug assistant prints the sent data packets and the servo status information, which corresponds to the data frames returned by the servo.



```
TX:FF FF FE 2 1 FE  
RX:FF FF 1 2 0 FC  
ID:1  
statu:0
```

3.2.1.3 Program Brief Analysis

- Import the servo communication library and command parsing library.

```
#include "HX_30HM.h"  
#include "stdio.h"  
#include "stdint.h"  
#include "Packet_Handler.h"
```

- Define the parameters to send and receive.

```
ServoStatus_t status;

int16_t write_pos = 4096;
int16_t read_pos = 4096;

int16_t write_pos_offset = 100;
int16_t read_pos_offset;

uint8_t write_acc = 100;

int16_t write_speed = 1000;
int16_t read_speed;

int16_t write_pwm_speed = 1000;

uint16_t write_torque = 1000;

int16_t sync_write1[2][4] = {{1, 0, 1000, 4095}, {2, 0, 1000, 4095}};
int16_t sync_write2[2][4] = {{1, 0, 1000, 0}, {2, 0, 1000, 0}};
uint8_t read_id[] = {1, 2};
int16_t sync_read_data[2][5];

uint8_t temp;
uint8_t vol;
uint16_t cur;
int16_t read_load;
uint8_t moving_status;
```

- Define two hardware serial ports: Serial 1 is used to print debug information to the PC serial debug assistant at a baud rate of 115200. Serial 2 is used for communication with the debugging board at a baud rate of 1000000.

```
#include "HX_30HM.h"

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_USART1_UART_Init();
MX_USART2_UART_Init();

/* Initialize interrupts */
MX_NVIC_Init();
/* USER CODE BEGIN 2 */
servo_init();
```

- The **ping** function broadcasts to search for servos, prints the detected ID numbers, and returns the working status of the servo.

```
status = servo_ping(0xFE);
```

- Print the sent data packets and the servo response frames to the PC serial debug assistant.

```

sprintf((char *)info_buf,"TX:%X %X %X %X %X %X\r\n",servo_packet.tx.header_1,\
servo_packet.tx.header_2,\
servo_packet.tx.elements.id,\
servo_packet.tx.elements.length,\
servo_packet.tx.elements.cmd,\

servo_packet.tx.elements.args[servo_packet.tx.elements.length - 2]\
);
serial_printf_string(info_buf);

sprintf((char *)info_buf,"RX:%X %X %X %X %X %X\r\n",servo_packet.rx.header_1,\
servo_packet.rx.header_2,\
servo_packet.rx.elements.id,\
servo_packet.rx.elements.length,\
servo_packet.rx.elements.cmd,\

servo_packet.rx.elements.args[servo_packet.rx.elements.length - 2]\
);
serial_printf_string(info_buf);

```

- Print the returned servo ID and operating status parameters to the PC serial debug assistant.

```

sprintf((char *)info_buf,"ID:%X\r\n",status.id);
serial_printf_string(info_buf);

sprintf((char *)info_buf,"statu:%X\r\n",status.error_byte);
serial_printf_string(info_buf);

```

3.2.2 Reading the Servo Status

This example displays the current position information of the bus servo through the serial debug assistant window.

3.2.2.1 Run Program

Double-click the **Serial_Servo.uvprojx** file located at the [STM32 Development \ Program \ Serial_Servo \ MDK-ARM](#) folder. Change the value of the **exp** macro definition in the **main.c** file to **1**, then compile the project.

```

/* Private define -----*/
/* USER CODE BEGIN PD */
#define exp 1

```

Connect the STM32F103RBT6 microcontroller to the computer and upload the compiled HEX file to the microcontroller.

3.2.2.2 Program Outcome

After running the program, the serial debug assistant prints the sent data packets and the current position of the servo, which corresponds to the data frames returned by the servo.

```
TX:FF FF 1 4 2 38 2 BE
RX:FF FF 1 4 0 45 A AB
Position:2629
```

3.2.2.3 Program Brief Analysis

- Import the servo communication library and command parsing library.

```
#include "HX_30HM.h"
#include "stdio.h"
#include "stdint.h"
#include "Packet_Handler.h"
```

- Define the parameters to send and receive.

```
ServoStatus_t status;

int16_t write_pos = 4096;
int16_t read_pos = 4096;

int16_t write_pos_offset = 100;
int16_t read_pos_offset;

uint8_t write_acc = 100;

int16_t write_speed = 1000;
int16_t read_speed;

int16_t write_pwm_speed = 1000;

uint16_t write_torque = 1000;

int16_t sync_write1[2][4] = {{1, 0, 1000, 4095}, {2, 0, 1000, 4095}};
int16_t sync_write2[2][4] = {{1, 0, 1000, 0}, {2, 0, 1000, 0}};
uint8_t read_id[] = {1, 2};
int16_t sync_read_data[2][5];

uint8_t temp;
uint8_t vol;
uint16_t cur;
int16_t read_load;
uint8_t moving_status;
```


- Define two hardware serial ports: Serial 1 is used to print debug information to the PC serial debug assistant at a baud rate of 115200. Serial 2 is used for communication with the debugging board at a baud rate of 1000000.

```
#include "HX_30HM.h"

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_USART1_UART_Init();
MX_USART2_UART_Init();

/* Initialize interrupts */
MX_NVIC_Init();
/* USER CODE BEGIN 2 */
servo_init();
```

- The `read_pos` function reads the current position value of servo ID 1.

```
status = servo_read_pos(1, &read_pos);
```

- Print the sent data packets and the servo response frames to the PC serial debug assistant.

```
printf((char *)info_buf, "TX:%X %X %X %X %X %X\r\n", servo_packet.tx.header_1, \
servo_packet.tx.header_2, \
servo_packet.tx.elements.id, \
servo_packet.tx.elements.length, \
servo_packet.tx.elements.cmd, \

servo_packet.tx.elements.args[servo_packet.tx.elements.length - 2] \
);
serial_printf_string(info_buf);

printf((char *)info_buf, "RX:%X %X %X %X %X %X\r\n", servo_packet.rx.header_1, \
servo_packet.rx.header_2, \
servo_packet.rx.elements.id, \
servo_packet.rx.elements.length, \
servo_packet.rx.elements.cmd, \

servo_packet.rx.elements.args[servo_packet.rx.elements.length - 2] \
);
serial_printf_string(info_buf);
```

- Print the returned current position values of the servo to the PC serial debug assistant.

```
printf((char *)info_buf, "Position:%hu\r\n", read_pos);
serial_printf_string(info_buf);
```

3.2.3 Servo Movement Control in Write Mode

3.2.3.1 Run Program

Double-click the **Serial_Servo.uvprojx** file located at the [STM32 Development \ Program \ Serial_Servo \ MDK-ARM](#) folder. Change the value of the **exp** macro definition in the **main.c** file to **1**, then compile the project.

```
/* Private define -----*/  
/* USER CODE BEGIN PD */  
#define exp 1
```

Connect the STM32F103RBT6 microcontroller to the computer and upload the compiled HEX file to the microcontroller.

3.2.3.2 Program Outcome

After running the program, the serial debugger prints the sent data packets and the data frames returned by the servo. The servo with the specified ID moves to the target position according to the configured parameters.

```
TX:FF FF 1 4 2 38 2 BE  
RX:FF FF 1 4 0 45 A AB  
Position:2629
```

3.2.3.3 Program Brief Analysis

- Import the servo communication library and command parsing library.

```
#include "HX_30HM.h"  
#include "stdio.h"  
#include "stdint.h"  
#include "Packet_Handler.h"
```

- Define the parameters to send and receive.

```
ServoStatus_t status;  
  
int16_t write_pos = 4096;  
int16_t read_pos = 4096;  
  
int16_t write_pos_offset = 100;  
int16_t read_pos_offset;  
  
uint8_t write_acc = 100;  
  
int16_t write_speed = 1000;  
int16_t read_speed;  
  
int16_t write_pwm_speed = 1000;
```

```
uint16_t write_torque = 1000;

int16_t sync_write1[2][4] = {{1, 0, 1000, 4095}, {2, 0, 1000, 4095}};
int16_t sync_write2[2][4] = {{1, 0, 1000, 0}, {2, 0, 1000, 0}};
uint8_t read_id[] = {1, 2};
int16_t sync_read_data[2][5];

uint8_t temp;
uint8_t vol;
uint16_t cur;
int16_t read_load;
uint8_t moving_status;
```

- Define two hardware serial ports: Serial 1 is used to print debug information to the PC serial debug assistant at a baud rate of 115200. Serial 2 is used for communication with the debugging board at a baud rate of 1000000.

```
#include "HX_30HM.h"

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_USART1_UART_Init();
MX_USART2_UART_Init();

/* Initialize interrupts */
MX_NVIC_Init();
/* USER CODE BEGIN 2 */
servo_init();
```

- The `write_pos` function sets the target position of servo ID 1, and the servo moves to that position.

```
status = servo_write_pos(1, write_pos);
```

- Print the sent data packets and the servo response frames to the PC serial debug assistant.

```
printf((char *)info_buf, "TX:%X %X %X %X %X %X\r\n", servo_packet.tx.header_1, \
servo_packet.tx.header_2, \
servo_packet.tx.elements.id, \
servo_packet.tx.elements.length, \
servo_packet.tx.elements.cmd, \

servo_packet.tx.elements.args[servo_packet.tx.elements.length - 2] \
);
serial_printf_string(info_buf);

printf((char *)info_buf, "RX:%X %X %X %X %X %X\r\n", servo_packet.rx.header_1, \
servo_packet.rx.header_2, \
servo_packet.rx.elements.id, \
servo_packet.rx.elements.length, \
servo_packet.rx.elements.cmd, \
```

```
servo_packet.rx.elements.args[servo_packet.rx.elements.length - 2] \
);
serial_printf_string(info_buf);
```

- Print the set target position values of the servo to the PC serial debug assistant.

```
sprintf((char *)info_buf, "Position:%hu\r\n", read_pos);
serial_printf_string(info_buf);
```

3.2.4 Reading the Servo Status

3.2.4.1 Run Program

Double-click the **Serial_Servo.uvprojx** file located at the [STM32 Development \ Program \ Serial Servo \ MDK-ARM](#) folder. Change the value of the **exp** macro definition in the **main.c** file to **1**, then compile the project.

```
/* Private define -----*/
/* USER CODE BEGIN PD */
#define exp 1
```

Connect the STM32F103RBT6 microcontroller to the computer and upload the compiled HEX file to the microcontroller.

3.2.4.2 Program Outcome

After running the program, the serial debugger prints the sent data packets and the data frames returned by the servo. The servo with the specified ID moves to the target position according to the configured parameters.

```
TX:FF FF 1 5 3 2A 0 10 BC
RX:FF FF 1 2 10 EC
Target_Position:4096
```

3.2.4.3 Program Brief Analysis

- Import the servo communication library and command parsing library.

```
#include "HX_30HM.h"
#include "stdio.h"
#include "stdint.h"
#include "Packet_Handler.h"
```

- Define the parameters to send and receive.

```
ServoStatus_t status;

int16_t write_pos = 4096;
int16_t read_pos = 4096;
```

```

int16_t write_pos_offset = 100;
int16_t read_pos_offset;

uint8_t write_acc = 100;

int16_t write_speed = 1000;
int16_t read_speed;

int16_t write_pwm_speed = 1000;

uint16_t write_torque = 1000;

int16_t sync_write1[2][4] = {{1, 0, 1000, 4095}, {2, 0, 1000, 4095}};
int16_t sync_write2[2][4] = {{1, 0, 1000, 0}, {2, 0, 1000, 0}};
uint8_t read_id[] = {1, 2};
int16_t sync_read_data[2][5];

uint8_t temp;
uint8_t vol;
uint16_t cur;
int16_t read_load;
uint8_t moving_status;

```

- The `write_reg_pos_ex` function writes the acceleration, speed, and position information to the specified register of servo ID 1 without executing immediately. Execution is deferred and requires calling the `reg_action` function.

```
status = servo_write_reg_pos_ex(1, write_acc, write_speed, write_pos);
```

- Print the sent data packets and the servo response frames to the PC serial debug assistant.

```

sprintf((char *)info_buf, "TX:%X %X %X %X %X %X %X %X %X %X %X %X %X\n",
servo_packet.tx.header_1, \
servo_packet.tx.header_2, \
servo_packet.tx.elements.id, \
servo_packet.tx.elements.length, \
servo_packet.tx.elements.cmd, \
servo_packet.tx.elements.args[0], \
servo_packet.tx.elements.args[1], \
servo_packet.tx.elements.args[2], \
servo_packet.tx.elements.args[3], \
servo_packet.tx.elements.args[4], \
servo_packet.tx.elements.args[5], \
servo_packet.tx.elements.args[6], \
servo_packet.tx.elements.args[7], \
servo_packet.tx.elements.args[8], \
servo_packet.tx.elements.args[servo_packet.tx.elements.length - 2] \
);
serial_printf_string(info_buf);

sprintf((char *)info_buf, "RX:%X %X %X %X %X %X\n", servo_packet.rx.header_1, \
servo_packet.rx.header_2, \

```

```
servo_packet.rx.elements.id,\
servo_packet.rx.elements.length,\
servo_packet.rx.elements.cmd,\
servo_packet.rx.elements.args[servo_packet.rx.elements.length - 2]\
);
serial_printf_string(info_buf);
```

- After calling `reg_action`, the servo executes the previously written `write_reg_pos_ex` command and moves accordingly.

```
status = servo_reg_action(1);
```

- Print the sent data packets and the servo response frames to the PC serial debug assistant.

```
printf((char *)info_buf, "TX:%X %X %X %X %X %X\r\n", servo_packet.tx.header_1,\
servo_packet.tx.header_2,\
servo_packet.tx.elements.id,\
servo_packet.tx.elements.length,\
servo_packet.tx.elements.cmd,\
servo_packet.tx.elements.args[servo_packet.tx.elements.length - 2]\
);
serial_printf_string(info_buf);

printf((char *)info_buf, "RX:%X %X %X %X %X %X\r\n", servo_packet.rx.header_1,\
servo_packet.rx.header_2,\
servo_packet.rx.elements.id,\
servo_packet.rx.elements.length,\
servo_packet.rx.elements.cmd,\
servo_packet.rx.elements.args[servo_packet.rx.elements.length - 2]\
);
serial_printf_string(info_buf);
```

- Print the set acceleration, speed, and position parameters to the PC serial debug assistant.

```
printf((char *)info_buf, "Target_Acc:%hu, Target_Speed:%hu, Target_Pos:%hu\r\n", write_acc,\
write_speed, write_pos);
serial_printf_string(info_buf);
```

3.2.5 Servo Movement Control in SyncWrite Mode

3.2.5.1 Run Program

Double-click the **Serial_Servo.uvprojx** file located at the [STM32 Development \ Program \ Serial Servo \ MDK-ARM](#) folder. Change the value of the **exp** macro definition in the **main.c** file to **1**, then compile the project.

```
/* Private define -----*/
/* USER CODE BEGIN PD */
#define exp 1
```

Connect the STM32F103RBT6 microcontroller to the computer and upload the compiled HEX file to the microcontroller.

3.2.5.2 Program Outcome

After running the program, the serial debugger prints the sent data packets. The servos with the specified IDs move to the target positions according to the configured parameters.

```
TX:FF FF 1 A 4 29 64 0 10 0 0 0 10 43 43
RX:FF FF 1 2 0 FC
TX:FF FF 1 2 5 F7
RX:FF FF 1 2 10 EC
Target_Acc:100, Target_Speed:1000, Target_Pos:4096
```

3.2.5.3 Program Brief Analysis

- Import the servo communication library and command parsing library.

```
#include "HX_30HM.h"
#include "stdio.h"
#include "stdint.h"
#include "Packet_Handler.h"
```

- Define the parameters to send and receive.

```
ServoStatus_t status;

int16_t write_pos = 4096;
int16_t read_pos = 4096;

int16_t write_pos_offset = 100;
int16_t read_pos_offset;

uint8_t write_acc = 100;

int16_t write_speed = 1000;
int16_t read_speed;

int16_t write_pwm_speed = 1000;

uint16_t write_torque = 1000;

int16_t sync_write1[2][4] = {{1, 0, 1000, 4095}, {2, 0, 1000, 4095}};
int16_t sync_write2[2][4] = {{1, 0, 1000, 0}, {2, 0, 1000, 0}};
uint8_t read_id[] = {1, 2};
int16_t sync_read_data[2][5];

uint8_t temp;
uint8_t vol;
uint16_t cur;
int16_t read_load;
```

```
uint8_t moving_status;
```

- Define two hardware serial ports: Serial 1 is used to print debug information to the PC serial debug assistant at a baud rate of 115200. Serial 2 is used for communication with the debugging board at a baud rate of 1000000.

```
#include "HX_30HM.h"

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_USART1_UART_Init();
MX_USART2_UART_Init();

/* Initialize interrupts */
MX_NVIC_Init();
/* USER CODE BEGIN 2 */
servo_init();
```

- The `sync_write_pos_ex` function sets the acceleration, speed, and position information for servo IDs 1 and 2, and the servos move to the specified positions.

```
// Write acceleration, speed, and position information to servo 1 and 2 simultaneously
status = servo_sync_write_pos_ex(sync_write1, 2);
```

- Print the sent data packets to the PC serial debug assistant.

```
sprintf((char *)info_buf, "Tx:%X %X %X %X %X %X %X %X %X %X %X %X %X %X %X %X %X %X %X %X %X\n", servo_packet.tx.header_1, \
servo_packet.tx.header_2, \
servo_packet.tx.elements.id, \
servo_packet.tx.elements.length, \
servo_packet.tx.elements.cmd, \
servo_packet.tx.elements.args[0], \
servo_packet.tx.elements.args[1], \

servo_packet.tx.elements.args[2], \
servo_packet.tx.elements.args[3], \
servo_packet.tx.elements.args[4], \
servo_packet.tx.elements.args[5], \
servo_packet.tx.elements.args[6], \
servo_packet.tx.elements.args[7], \
servo_packet.tx.elements.args[8], \
servo_packet.tx.elements.args[9], \
servo_packet.tx.elements.args[10], \

servo_packet.tx.elements.args[11], \
servo_packet.tx.elements.args[12], \
servo_packet.tx.elements.args[13], \
servo_packet.tx.elements.args[14], \
servo_packet.tx.elements.args[15], \
```



```
servo_packet.tx.elements.args[16],\
servo_packet.tx.elements.args[17],\
servo_packet.tx.elements.args[18],\
servo_packet.tx.elements.args[19],\
servo_packet.tx.elements.args[servo_packet.tx.elements.length - 2]\

);
serial_printf_string(info_buf);
```

- Print the set servo ID, acceleration, speed, and position parameters to the PC serial debug assistant.

```
sprintf((char*)info_buf,"ServoID:%hu,Target_Acc:%hu,Target_Speed:%hu,Target_Pos:%hu\r\n",\
sync_write1[0][0],sync_write1[0][1],\
sync_write1[0][2],sync_write1[0][3]);
serial_printf_string(info_buf);

sprintf((char
*)info_buf,"ServoID:%hu,Target_Acc:%hu,Target_Speed:%hu,Target_Pos:%hu\r\n",\
sync_write1[1][0],sync_write1[1][1],\
sync_write1[1][2],sync_write1[1][3]);
serial_printf_string(info_buf);
```

3.2.6 Reading the Servo Status

3.2.6.1 Run Program

Double-click the **Serial_Servo.uvprojx** file located at the [STM32 Development \ Program \ Serial_Servo \ MDK-ARM](#) folder. Change the value of the **exp** macro definition in the **main.c** file to **1**, then compile the project.

```
/* Private define -----*/
/* USER CODE BEGIN PD */
#define exp 1
```

Connect the STM32F103RBT6 microcontroller to the computer and upload the compiled HEX file to the microcontroller.

3.2.6.2 Program Outcome

After running the program, the serial debugger prints the sent data packets and the data frames returned by the servo. The operating mode of the servo with the specified ID is set.

```
TX:FF FF FE 14 83 29 7 1 0 FF F 0 0 E8 3 2 0 FF F 0 0
ServoID:1,Target_Acc:0,Target_Speed:1000,Target_Pos:4095
ServoID:2,Target_Acc:0,Target_Speed:1000,Target_Pos:4095
```

3.2.6.3 Program Brief Analysis

- Import the servo communication library and command parsing library.

```
#include "HX_30HM.h"
#include "stdio.h"
#include "stdint.h"
#include "Packet_Handler.h"
```

- Define the parameters to send and receive.

```
ServoStatus_t status;

int16_t write_pos = 4096;
int16_t read_pos = 4096;

int16_t write_pos_offset = 100;
int16_t read_pos_offset;

uint8_t write_acc = 100;

int16_t write_speed = 1000;
int16_t read_speed;

int16_t write_pwm_speed = 1000;

uint16_t write_torque = 1000;

int16_t sync_write1[2][4] = {{1, 0, 1000, 4095}, {2, 0, 1000, 4095}};
int16_t sync_write2[2][4] = {{1, 0, 1000, 0}, {2, 0, 1000, 0}};
uint8_t read_id[] = {1, 2};
int16_t sync_read_data[2][5];

uint8_t temp;
uint8_t vol;
uint16_t cur;
int16_t read_load;
uint8_t moving_status;
```

- Define two hardware serial ports: Serial 1 is used to print debug information to the PC serial debug assistant at a baud rate of 115200. Serial 2 is used for communication with the debugging board at a baud rate of 1000000.

```
#include "HX_30HM.h"

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_USART1_UART_Init();
MX_USART2_UART_Init();

/* Initialize interrupts */
MX_NVIC_Init();
/* USER CODE BEGIN 2 */
servo_init();
```

- The `select_mode` function sets the operating mode of servo ID 1.

```
status = servo_select_mode(1, 0);
```

- Print the sent data packets and the servo response frames to the PC serial debug assistant.

```
printf((char *)info_buf, "TX:%X %X %X %X %X %X %X %X\r\n", servo_packet.tx.header_1, \
servo_packet.tx.header_2, \
servo_packet.tx.elements.id, \

servo_packet.tx.elements.length, \
servo_packet.tx.elements.cmd, \

servo_packet.tx.elements.args[0], \

servo_packet.tx.elements.args[1], \
servo_packet.tx.elements.args[servo_packet.tx.elements.length - 2] \
);
serial_printf_string(info_buf);

printf((char *)info_buf, "RX:%X %X %X %X %X %X\r\n", servo_packet.rx.header_1, \
servo_packet.rx.header_2, \
servo_packet.rx.elements.id, \
servo_packet.rx.elements.length, \
servo_packet.rx.elements.cmd, \
servo_packet.rx.elements.args[servo_packet.rx.elements.length - 2] \
);
serial_printf_string(info_buf);
```

- Print the returned servo ID and operating status parameters to the PC serial debug assistant.

```
printf((char *)info_buf, "Position:%hu\r\n", read_pos);
serial_printf_string(info_buf);
```